

Software Architecture Document
SAD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SAD
Versión	1.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — juanromeroo@javeriana.edu.co
Laura Juliana Garzón Arias — garzonlj@javeriana.edu.co
Xiomara Herrera Suárez — x_herrera@javeriana.edu.co
Sebastián Almanza Galvis — s.almanzag@javeriana.edu.co

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.

Trazabilidad. Este SAD define la arquitectura del sistema y traza su relación con el SRS de GammaLab v2.0.

Índice

1. Introducción	6
1.1. Propósito	6
1.2. Alcance	6
1.3. Definiciones y acrónimos	6
1.4. Audiencia	7
1.5. Referencias relacionadas	7
2. Contexto del sistema	7
2.1. Visión general	7
2.2. Límites del sistema	8
2.2.1. Dentro del alcance	8
2.2.2. Fuera del alcance	8
2.2.3. Entradas externas	8
2.2.4. Salidas externas	8
3. Metas y Restricciones de la Arquitectura	9
3.1. Requisitos de los Stakeholders	9
3.2. Atributos de Calidad	9
3.3. Supuestos y limitaciones derivadas	11
3.3.1. Restricciones de arquitectura	11
3.3.2. Supuestos de operación (Evolución de v1.0)	11
4. Principios y decisiones arquitectónicas	11
4.1. Estilo base	11
4.1.1. Orquestador del sistema	13
4.1.2. Componentes del núcleo	13
4.1.3. Arquitectura basada en plugins	14
4.1.4. Principio de desacoplamiento y extensibilidad	14
4.1.5. Justificación	15
4.1.6. Decisiones arquitectónicas	15
4.1.7. Patrones utilizados	16
5. Vistas Arquitectónicas (4+1)	17
5.1. Objetivo	17
5.2. Vista de Escenarios	18

5.3.	Vista Lógica	20
5.3.1.	Propósito	20
5.3.2.	Notación	20
5.3.3.	Visión general del modelo	20
5.3.4.	Elementos del núcleo	21
5.3.5.	Servicios del sistema	21
5.3.6.	Elementos de presentación	23
5.3.7.	Contrato de extensión	23
5.3.8.	Extensiones	24
5.3.9.	Modelo de datos	24
5.3.10.	Relaciones estructurales	25
5.3.11.	Justificación de las decisiones	25
5.3.12.	Trazabilidad con los requerimientos	26
5.4.	Vista de Procesos	26
5.5.	Vista Física	28
6.	Atributos de calidad	29
6.1.	Escenarios de calidad	29
6.1.1.	Escenario de Calidad 1: Integración de un Plugin Externo	29
6.1.2.	Escenario de Calidad 2: Carga de Señal de Gran Tamaño	30
6.1.3.	Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución	30
6.1.4.	Escenario de Calidad 4: Exportación de Datos o Gráficas	31
6.1.5.	Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica	31
6.1.6.	Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas	32
6.1.7.	Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles	32
6.1.8.	Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas	33
6.2.	Reglas de compatibilidad y versionado	33
6.3.	Métricas y plan de verificación	34
6.4.	Plan de evidencia	35
6.5.	Conclusión	35
7.	Anexos	36
7.1.	Anexo A — Relación con el SRS	36
7.2.	Anexo B — Relación con el SDD	36
7.3.	Anexo C — Trazabilidad global de artefactos	36
7.4.	Anexo D — Diagramas y tablas de referencia	36

8. Referencias

36

Índice de figuras

1.	Diagrama conceptual de la arquitectura microkernel de GammaLab 2.0, mostrando la relación entre el núcleo central, los servicios compartidos y los plugins especializados.	12
2.	Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones <i>include/extend</i> se tomaron literalmente del campo “Extensiones” de cada ficha; ninguna es inferida por similitud temática.	18
3.	Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados.	21
4.	Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.	27
5.	Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.	28

Índice de cuadros

1.	Acrónimos y términos utilizados en el documento	6
2.	Stakeholders principales y expectativas	9
3.	ADRs resumidas: decisión, motivación y consecuencias.	16
4.	Patrones arquitectónicos aplicados en GammaLab 2.0.	17
5.	UC-01 — Calcular acoplamiento fase-amplitud (PAC) de un canal	18
6.	UC-02 — Cancelar un cómputo pesado en curso	19
7.	UC-03 — Guardar y reabrir un proyecto (.glab)	19
8.	Extensiones representadas en la vista lógica.	24
9.	Relaciones estructurales de la vista lógica.	25
10.	Trazabilidad entre elementos de la vista lógica y requerimientos.	26
11.	Escenario de calidad 1 — Integración de un plugin externo	29
12.	Escenario de calidad 2 — Carga de señal de gran tamaño	30
13.	Escenario de calidad 3 — Manejo de errores en tiempo de ejecución	30
14.	Escenario de calidad 4 — Exportación de datos o gráficas	31
15.	Escenario de calidad 5 — Operación por un usuario sin experiencia técnica	31
16.	Escenario de calidad 6 — Rendimiento con múltiples señales cargadas	32
17.	Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles	32
18.	Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas	33
19.	Versiones de dependencias externas ancladas en requirements.txt	34
20.	Métricas, método de verificación y criterio de aceptación por atributo de calidad	34

1. Introducción

1.1. Propósito

El presente documento describe la arquitectura de referencia del sistema GammaLab 2.0 y establece los principios, decisiones y lineamientos que orientan su diseño arquitectónico evolutivo. Expone las diferentes vistas arquitectónicas bajo el modelo 4+1 y la representación de contexto del modelo C4, proporcionando una comprensión integral de la estructura modular, los componentes y sus integraciones dinámicas. Buscar servir como un contrato técnico entre los equipos de desarrollo, validación y demás stakeholders, garantizando una base común para la toma de decisiones, la mitigación de errores y la futura evolución del sistema. Sin contar que define los criterios necesarios para asegurar que los atributos de calidad sean evaluados y cumplirse de manera medible.

1.2. Alcance

Este documento abarca la definición de los principios y decisiones arquitectónicas que rigen la estructura de GammaLab v2.0, destacando el desacoplamiento de su interfaz de usuario mediante un kernel asíncrono y su extendibilidad mediante plugins. Asimismo, desarrolla las vistas lógicas, físicas, de desarrollo y de procesos del sistema, e identifica las estrategias de mitigación para riesgos críticos como el control de concurrencia en Python y el consumo acumulativo de memoria RAM. Finalmente, se definen los escenarios de calidad para la validación del sistema y su trazabilidad detallada con la especificación del SRS v2.0. Por fuera del alcance de este documento quedan las políticas de gobernanza del repositorio público de código abierto, el despliegue automatizado en entornos de integración continua (CI/CD) y las guías detalladas para el usuario final, las cuales se consolidan en el Manual de Usuario.

1.3. Definiciones y acrónimos

Cuadro 1: Acrónimos y términos utilizados en el documento

Sigla / Término	Definición
SAD	Software Architecture Document. Documento de arquitectura del sistema.
SRS	Software Requirements Specification. Documento de especificación de requisitos.
SDD	Software Design Document. Documento de diseño técnico.
IoC	Inversión de Control. Principio que delega el flujo de ejecución a un contenedor.
DI	Dependency Injection. Técnica para resolver dependencias desde un contenedor externo.
MVP / MVC	Patrones de separación de interfaz y lógica (Model–View–Presenter / Model–View–Controller).
Template Method	Patrón que define un flujo general con pasos sobrescribibles.
Adapter	Patrón para estandarizar interfaces incompatibles.
Caching	Estrategia para evitar recalcular o recomponer estructuras de datos.
Python	Lenguaje de programación principal de <i>Gamma Lab</i> .
PyQt5	Framework de interfaz gráfica basado en Qt para Python.

Sigla / Término	Definición
VTK	Visualization Toolkit. Biblioteca para visualización científica 2D/3D.
QVTK	Interfaz <code>QVTKRenderWindowInteractor</code> para incrustar VTK en aplicaciones Qt.
NumPy	Librería para cálculo numérico vectorizado.
EDF	European Data Format. Formato estándar para señales biomédicas.
ABF	Axon Binary File. Formato de archivo utilizado por Axon Instruments para registros biomédicos.
GUI Toolkit	Conjunto de herramientas para construir interfaces gráficas (Qt, PyQt, VTK).

1.4. Audiencia

Este documento está dirigido al equipo de desarrollo, así como a cualquier stakeholder con formación técnica que requiera comprender las decisiones arquitectónicas del sistema. Asimismo, está orientado a todos los investigadores y personas del Laboratorio de Electrofisiología de la Universidad de Colombia que estén interesados en conocer la estructura interna, los principios de diseño y los mecanismos de GammaLab v2.0 para el uso, mantenimiento y futuros cambios.

1.5. Referencias relacionadas

Este SAD se alinea con:

- **SRS de Gamma Lab:** requisitos funcionales y no funcionales del sistema.

Las referencias técnicas externas (PyQt, VTK, NumPy, EDF/ABF, YAML).

2. Contexto del sistema

2.1. Visión general

GammaLab v2.0 está estructurado como una aplicación de escritorio tipo standalone diseñada para ser ejecutada de forma local, sin requerir conectividad de red. Su arquitectura se basa en un patrón modular de microkernel, donde un núcleo central asíncrono (desarrollado en Python con el framework gráfico PySide6 y la librería de visualización científica VTK) orquesta la ejecución y comunicación de plugins independientes a través de un almacén de datos en memoria (DataStore). La evolución fundamental de esta segunda iteración se muestra una evolución de los hilos únicos de Python, mediante la incorporación de estrategias de paralelización basadas en multiprocesamiento y multithreading en el kernel. Esto permite que el sistema distribuya las tareas pesadas (como la transformada Wavelet y el acoplamiento cruzado de frecuencias) sin congelar la interfaz de usuario.

2.2. Límites del sistema

2.2.1. Dentro del alcance

- **Ingreso y lectura:** Carga nativa de registros electroencefalográficos (EEG) almacenados localmente en formatos estándares .edf, .abf, .bdf y .mat.
- **Procesamiento y segmentación:** Aplicación de filtros digitales de corte, remoción de artefactos de estimulación y segmentación de señales en ensayos (trials) mediante umbrales en mV u operaciones de intervalo interestímulo (ISI).
- **Procesamiento espectral y tiempo-frecuencia:** Cálculo de promedios de Potenciales Relacionados a Eventos (ERP) con visualización de Butterfly plot bajo demanda, análisis de Transformada Rápida de Fourier (FFT), Densidad Espectral de Potencia (PSD) y Transformada Wavelet Continua con normalización y escalas personalizables.
- **Análisis de conectividad y acoplamiento avanzado:** Ejecución del análisis de acoplamiento fase-amplitud (PAC) tanto de un canal como cruzado intercanal.
- **Persistencia del espacio de trabajo:** Guardado y restauración del estado de la sesión analítica mediante archivos de proyecto local con extensión personalizada .glab.
- **Exportación de resultados:** Generación de archivos de datos numéricos estructurados (CSV con ejes X simplificados, Excel, JSON) y representaciones gráficas de alta resolución, incluyendo formato de imagen estándar (PNG, JPG, TIFF) e imagen vectorial científica (EMF).

2.2.2. Fuera del alcance

- La comunicación o adquisición de señales EEG en tiempo real mediante conexiones directas a equipos de amplificación, electrodos o diademas inalámbricas.
- La persistencia de proyectos en bases de datos centralizadas, servicios en la nube, servidores remotos o API web distribuidas; toda la persistencia es local e individual.
- El soporte operativo para plataformas web, sistemas móviles o sistemas operativos Unix/macOS en distribución comercial nativa (el instalador standalone se restringe a Windows de 64 bits).
- Mecanismos de control de acceso multiusuario, autenticación de seguridad en red, cifrado de datos clínicos o sincronización colaborativa síncrona.

2.2.3. Entradas externas

- Señales neurofisiológicas digitalizadas en formatos binarios compatibles (.edf, .abf, .bdf, .mat).
- Configuración alfanumérica de entrada por el usuario clínico (frecuencias de muestreo, umbrales de trial en mV, bandas de frecuencia de fase/amplitud para PAC/PLV y ciclos de wavelet).
- Eventos de interacción directa del investigador con la GUI mediante teclado y ratón para realizar zoom multidireccional, desplazamiento temporal (pan) y selección de puntos específicos sobre las gráficas para la toma de medidas.

2.2.4. Salidas externas

- Gráficas 2D renderizadas mediante VTK embebido en PySide6, tales como comodulogramas de PAC, mapas de calor de coherencia temporal-frecuencial y espectrogramas Wavelet.

- Archivos con metadatos y estados serializados .glab que contienen los análisis intermedios, rutas absolutas y configuraciones del proyecto.
- Tablas de resultados analíticos en formato Excel (.xlsx), JSON y archivos CSV optimizados para evitar redundancia de ejes temporales.
- Archivos de imagen estándar (PNG/JPG/TIFF) y gráficos vectoriales de alta precisión (EMF) para uso directo en publicaciones indexadas.

3. Metas y Restricciones de la Arquitectura

3.1. Requisitos de los Stakeholders

Cuadro 2: Stakeholders principales y expectativas

Stakeholder	Tipo	Necesidades / Intereses
Laboratorio de Electrofisiología de la Universidad de Colombia	Usuario final	Su interés principal es contar con una herramienta robusta, intuitiva y libre de licencias que simplifique el análisis complejo por medio de PAC, sin necesitar habilidades de programación.
Equipo de desarrollo	Interno	Implementar una arquitectura extensible (microkernel + plugins), reestructurar el diseño de las pantallas aplicando estándares de codificación limpia y utilizar paralelización asíncrona.
Directora de tesis	Interno	Validar la rigurosidad científica de los algoritmos de frecuencia y supervisar el cumplimiento del alcance metodológico.
Evaluable académicos	Externo / Validador	Verificar el cumplimiento de requerimientos, funciones y metodología aplicada desde la ingeniería de sistemas e ingeniería electrónica, demostrando cuantitativamente la mejora de rendimiento con respecto a GammaLab v1.0.
Comunidad científica	Externo / Colaborador	Posibilidad de reutilizar y mejorar el código abierto; transparencia metodológica y reproducibilidad de resultados; estandarización de flujos de trabajo y formatos de análisis EEG.

3.2. Atributos de Calidad

La arquitectura de **GammaLab 2.0** fue diseñada pensando principalmente en las necesidades manifestadas en el laboratorio de electrofisiología, entendiendo sus flujos de trabajo y los requerimientos propios de la ejecución de análisis experimentales. Al mismo tiempo, se proyecta como un software adaptable a distintos contextos y paradigmas de investigación, por lo que resulta fundamental considerar el impacto de los siguientes atributos de calidad en esta segunda versión:

1. **Extensibilidad:** Dado que el sistema debe adaptarse tanto a diferentes tipos de experimentos como a diversos entornos de investigación, la extensibilidad se concibe como una capacidad primordial. El diseño permite agregar nuevos módulos personalizados, completamente programables por cada laboratorio o usuario, y ofrece una interfaz clara para integrarlos

de manera sencilla dentro del sistema. En **GammaLab 2.0**, esta capacidad se optimiza al desacoplar los plugins del núcleo mediante un contenedor de **Inversión de Control (IoC)**, registrando servicios comunes (como **ExportService**, **MeasurementService** y **SettingsService**) que evitan importaciones directas o acoplamientos rígidos. Asimismo, la entidad **SignalDataset** expone una API pública de comunicación estricta que encapsula la manipulación de trials, garantizando que futuras extensiones no alteren la integridad de los datos en memoria.

2. **Usabilidad:** GammaLab se privilegia de un diseño de interfaz de usuario pensado para ser lo más sencillo y entendible posible en cada funcionalidad implementada. Lo anterior resulta especialmente importante en el caso de los plugins, donde se manejan procesos conceptualmente complejos. En esta nueva versión, se eliminan las barreras técnicas de uso mediante ayudas contextuales integradas (*tooltips*) para todos los parámetros, menús unificados de edición en la vista principal (*Home*) y la representación explícita de gráficas complejas como el *Butterfly plot*. Adicionalmente, el sistema asegura la retroalimentación constante del estado del sistema mediante barras de progreso no bloqueantes para operaciones superiores a 1 segundo, garantizando un flujo intuitivo y controlado.
3. **Mantenibilidad:** La arquitectura propuesta favorece la modularidad y el ordenamiento claro del código, buscando que el sistema siga principios como el de Abierto/Cerrado dentro del marco SOLID: abierto a extensiones y cerrado a modificaciones innecesarias. El sistema realiza una separación rigurosa entre la interfaz gráfica (PySide6) y la lógica matemática en hilos de cómputo independientes, facilitando la depuración del código. Además, la mantenibilidad y la robustez técnica se garantizan mediante el apego estricto a las directrices de estilo **PEP8** y un estándar mínimo del **80 % de cobertura de pruebas unitarias automatizadas** utilizando el framework **Pytest**.
4. **Rendimiento:** El contexto de uso del programa exige un rendimiento estable y un consumo controlado de recursos, especialmente dada la naturaleza de los datos involucrados, que suelen tener alta densidad de información. Para dar soporte estable a señales de hasta 600 MB en memoria, **GammaLab 2.0** incorpora una estrategia de paralelización en el Kernel. Las tareas pesadas de procesamiento (PAC, Wavelet, PSD) se distribuyen de forma asíncrona mediante multiprocesamiento (**multiprocessing**), impidiendo que la interfaz gráfica se bloquee durante los análisis (manteniendo la responsividad ≤ 0.3 s). Adicionalmente, se introduce el submuestreo dinámico visual a un límite de 2000 puntos por serie en los widgets de VTK para conservar una navegación fluida (≥ 24 FPS) sin degradar la data cruda, acompañado de rutinas explícitas de liberación de memoria de GPU y RAM en cada plugin al cambiar de módulo.
5. **Portabilidad:** El sistema debe asegurar su correcta ejecución en diversos entornos locales sin requerir la intervención técnica continua del equipo desarrollador. **GammaLab 2.0** centraliza su portabilidad local en sistemas Windows 10/11 de 64 bits. Este atributo se consolida al empaquetar la aplicación de manera autónoma en un **ejecutable standalone (.exe)** a través de PyInstaller. Esto permite que el personal clínico e investigadores ejecuten análisis avanzados localmente con un doble clic, omitiendo por completo la configuración o instalación manual de librerías científicas y consolas de Python en sus estaciones de trabajo.
6. **Robustez y Fiabilidad:** El sistema debe mantener la estabilidad operativa en sesiones continuas y tolerar errores imprevistos en tiempo de ejecución. Ante excepciones internas generadas por plugins de terceros o formatos de archivo corruptos, el sistema captura los fallos de manera centralizada sin interrumpir la sesión del usuario ni provocar la caída de la aplicación. Adicionalmente, el sistema implementa exclusiones mutuas para accesos concurrentes al almacén de datos centralizado (*DataStore*), asegura la correcta liberación

de observadores y callbacks VTK y proporciona terminaciones controladas (*cleanup*) de hilos de procesamiento de datos ante solicitudes explícitas de cancelación.

7. **Trazabilidad y Persistencia:** El sistema incorpora en cada análisis la información contextual y los metadatos necesarios para asegurar la reproducibilidad y trazabilidad de los resultados. **GammaLab 2.0** materializa esta capacidad de forma completa mediante el módulo **PJ (Project Session)**, el cual introduce la persistencia local del estado completo del espacio de trabajo (rutas de señales, trials configurados y análisis procesados) en archivos con extensión de proyecto **.glab** o **.glws**. Asimismo, la trazabilidad científica externa se respalda al exportar resultados consistentes en formatos tabulares (CSV sin redundancia de ejes temporales, Excel, JSON) y gráficos de alta calidad vectorial en formato **EMF**.

1.

3.3. Supuestos y limitaciones derivadas

A continuación se presentan los supuestos y limitaciones considerados durante el diseño y construcción del sistema:

3.3.1. Restricciones de arquitectura

- **Operación standalone y offline:** El sistema debe ejecutarse de forma local en la máquina del investigador de forma estricta. No se permite el uso de servicios en la nube, APIs de procesamiento externas ni el uso de bases de datos distribuidas.
- **Entorno tecnológico base:** El núcleo matemático y la lógica de interacción deben desarrollarse bajo el lenguaje Python (versión mínima 3.11), utilizando Pyside6 para la interfaz y VTK para la visualización científica.
- **Compatibilidad de datos:** El módulo de entrada/salida soporta y procesa archivos de señales en formatos como: EDF, ABF, BDF, MAT y GLAB.

3.3.2. Supuestos de operación (Evolución de v1.0)

- **Procesamiento concurrente:** En esta nueva versión 2.0 se asume que el hardware cuenta con arquitectura multinúcleo (más de 4 núcleos) para el despliegue de procesamiento asíncrono y multiprocesamiento en segundo plano.
- **Naturaleza del proyecto:** El sistema está diseñado exclusivamente para el posprocesamiento y análisis de señales previamente adquiridas. No soporta adquisición de datos en tiempo real ni comunicación con hardware de amplificación o registro fisiológico.

4. Principios y decisiones arquitectónicas

4.1. Estilo base

La arquitectura de GammaLab 2.0 adopta un estilo microkernel, en el que el sistema central o *core system* proporciona los servicios esenciales de comunicación, gestión de plugins, registro de eventos, coordinación de tareas y manejo de datos. Este núcleo actúa como mediador entre los

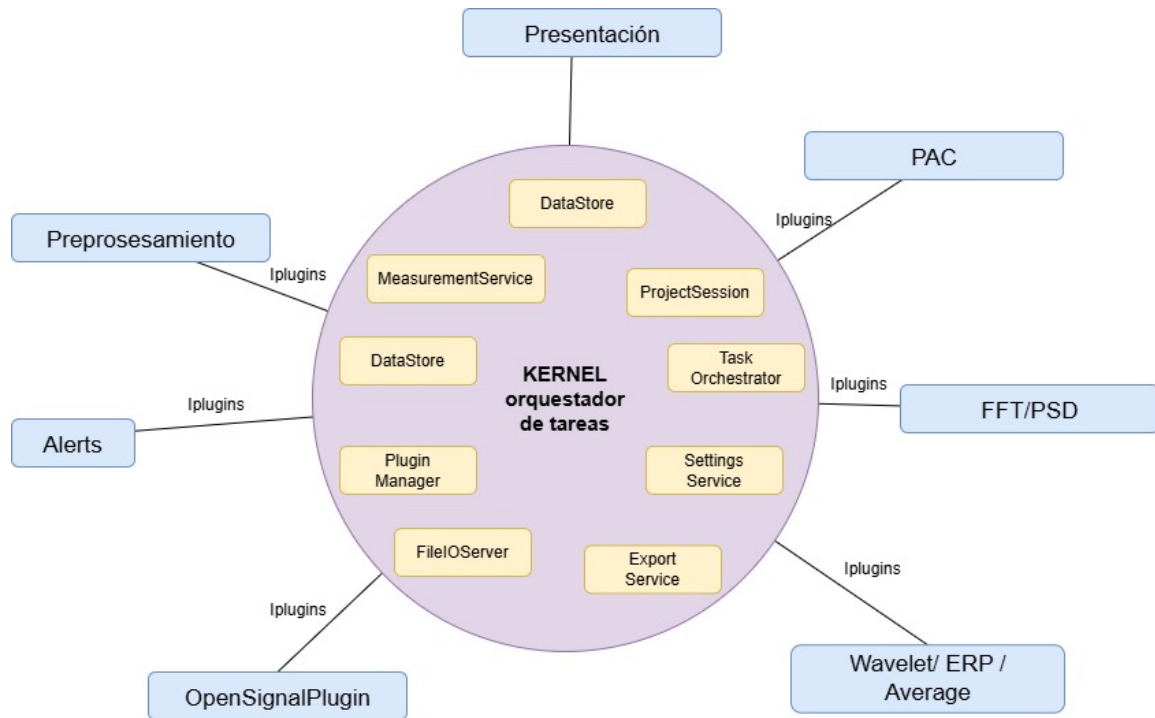


Figura 1: Diagrama conceptual de la arquitectura microkernel de GammaLab 2.0, mostrando la relación entre el núcleo central, los servicios compartidos y los plugins especializados.

distintos módulos funcionales del sistema, permitiendo que cada plugin se conecte e interactúe mediante una interfaz común, sin acoplarse directamente a la lógica interna del sistema.

El *Kernel* es el componente central de esta arquitectura y desempeña la función de orquestador principal. Su responsabilidad consiste en coordinar la ejecución de tareas, gestionar la cola de trabajos, supervisar el estado de cada operación, controlar la cancelación de procesos en ejecución y administrar la sincronización entre la interfaz gráfica y los módulos de análisis. De esta forma, la capa de presentación permanece responsiva mientras el procesamiento de señales y cálculos estadísticos se ejecutan en segundo plano.

La separación entre la lógica estable del sistema y la funcionalidad específica de análisis es una de las decisiones más relevantes del diseño. El *core* encapsula los servicios transversales de la aplicación, como el almacenamiento de datos, la gestión de eventos, la carga de archivos, la exportación de resultados y la configuración del entorno operativo. Por su parte, los plugins especializados implementan las distintas capacidades del sistema, como FFT, PSD, Wavelet, ERP, PAC y otras herramientas de análisis.

Esta estructura favorece la extensibilidad del sistema, ya que permite incorporar nuevos módulos funcionales sin modificar la lógica del núcleo. Además, reduce el acoplamiento entre componentes y mejora la mantenibilidad del proyecto a lo largo de su evolución. La relación entre el *core* y los plugins se establece a través de una API de integración basada en la interfaz *IPlugin*, la cual define el contrato que cada módulo debe cumplir para incorporarse al ecosistema de GammaLab 2.0.

En este sentido, la arquitectura del sistema se organiza en tres capas principales: la capa de presentación, donde se encuentra la interfaz gráfica desarrollada con PySide6 y Qt; la capa de coordinación, representada por el Kernel y los servicios compartidos del core; y la capa funcional, conformada por los plugins que implementan los distintos análisis y visualizaciones.

4.1.1. Orquestador del sistema

El Kernel de GammaLab 2.0 se define como el orquestador central del sistema. Su principal función es coordinar la ejecución de tareas complejas y garantizar que las operaciones intensivas de cómputo no bloqueen la interfaz gráfica. Esta decisión es clave para el rendimiento del sistema, ya que el análisis de señales EEG implica el manejo de datos de alta resolución, filtros, transformadas, estimaciones de potencia y análisis de acoplamiento cruzado de frecuencias.

Entre las funciones del orquestador se encuentran:

- Registro y descubrimiento de plugins.
- Gestión de la cola de tareas y priorización de ejecución.
- Distribución de tareas entre hilos o procesos según su complejidad.
- Control del estado de cada operación: pendiente, ejecutándose, completada o cancelada.
- Supervisión del progreso de los análisis.
- Coordinación con los servicios del core.
- Entrega de resultados hacia la capa visual.
- Gestión de errores y alertas a la interfaz de usuario.

El orquestador actúa como el punto de integración entre la capa de presentación, los servicios del sistema y los módulos especializados. Cuando el usuario solicita ejecutar un análisis, la interfaz no invoca directamente la lógica del plugin; en cambio, la solicitud se envía al Kernel, quien valida el contexto, identifica el módulo apropiado, crea la tarea y la ejecuta de forma controlada. Esta estructura evita duplicación de lógica, reduce fallos de sincronización y mejora la trazabilidad de las operaciones.

En términos arquitectónicos, el Kernel también centraliza la administración de recursos del sistema. Esto incluye la gestión de la memoria compartida, la preparación de datos para análisis y la actualización del estado global del entorno. En un sistema orientado a señales neurofisiológicas, donde los volúmenes de información pueden ser elevados, esta coordinación es crítica para asegurar estabilidad, eficiencia y capacidad de respuesta.

4.1.2. Componentes del núcleo

El núcleo de GammaLab 2.0 está conformado por una serie de componentes que permiten la operación global del sistema. Estos módulos no son reemplazables por plugins, sino que constituyen la base funcional sobre la cual todo el entorno opera.

Los principales componentes del core son:

- **Kernel:** responsable de la orquestación y coordinación de tareas.
- **PluginManager:** encargado de registrar, validar y cargar los plugins disponibles.
- **DataStore:** almacena señales, resultados intermedios, metadatos, configuraciones y sesión activa.
- **FileIOService:** gestiona la lectura y validación de archivos EEG en formatos como ABF, EDF, BDF y MAT.

- **MeasurementService**: administra métricas, cálculos y resultados derivados del análisis.
- **SettingsService**: maneja la configuración general del sistema y preferencias del usuario.
- **ExportService**: se encarga de la exportación de resultados en formatos compatibles con análisis e informes.
- **ProjectSession**: gestiona la persistencia del espacio de trabajo mediante archivos de proyecto `.glab`, guardando y restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.

El diseño del núcleo sigue un principio de servicio compartido, donde los plugins no crean instancias aisladas de cada funcionalidad, sino que acceden a los servicios del sistema mediante interfaces controladas por el Kernel. Este enfoque reduce el acoplamiento entre módulos y facilita la evolución del sistema sin afectar la lógica base.

4.1.3. Arquitectura basada en plugins

GammaLab 2.0 implementa una arquitectura modular basada en plugins, con el objetivo de ampliar la funcionalidad del sistema sin modificar el núcleo central. Cada plugin encapsula una capacidad específica de análisis o visualización, y se integra con la aplicación mediante un contrato común definido por la interfaz *IPlugin*.

Un plugin de GammaLab 2.0 puede contener:

- lógica de procesamiento,
- componentes de interfaz gráfica,
- configuraciones específicas del análisis,
- metadatos de identificación,
- conexión con el DataStore y el Kernel.

El *PluginManager* realiza el descubrimiento dinámico de nuevos módulos, validando la presencia de metadatos y registrando cada extensión dentro del ecosistema del sistema. Esta estructura permite que el sistema evolucione de forma incremental: cada nuevo módulo puede añadirse a la aplicación sin recompilar el núcleo ni modificar la lógica general del software.

4.1.4. Principio de desacoplamiento y extensibilidad

La arquitectura de GammaLab 2.0 se fundamenta en el principio de desacoplamiento funcional. La capa de presentación, el Kernel y los plugins mantienen una separación clara de responsabilidades. Esta decisión permite que cada módulo evolucione de forma independiente sin comprometer la estabilidad general del sistema. Por ejemplo, una modificación en la interfaz de usuario no exige cambiar la lógica del análisis; un nuevo módulo de procesamiento puede agregarse sin alterar la estructura del núcleo; y una actualización en la gestión de archivos no afecta directamente a la visualización de resultados.

La extensibilidad también se ve reforzada por un conjunto de mecanismos bien definidos:

- Registro dinámico de plugins.

- Contrato de integración mediante IPlugin.
- Gestión centralizada de datos mediante DataStore.
- Coordinación de tareas mediante el Kernel.
- Desacoplamiento entre lógica funcional y visualización.

4.1.5. Justificación

La arquitectura adoptada para GammaLab 2.0 corresponde a un estilo microkernel, en el que el núcleo central del sistema es el core y, dentro de este, el Kernel cumple la función de orquestador principal. Esta decisión permite separar la lógica estable del sistema de las funcionalidades específicas de análisis, manteniendo un diseño modular, extensible y de bajo acoplamiento.

El Kernel centraliza la coordinación de tareas, la gestión de la cola de ejecución, la supervisión del estado de cada operación y la delegación de la lógica funcional a los plugins. Esta decisión es necesaria porque el procesamiento de señales EEG requiere operaciones computacionalmente intensivas, como PAC, FFT, PSD y Wavelet, que no deben ejecutarse directamente en la interfaz gráfica. Además, al separar la lógica de coordinación del procesamiento funcional, el sistema mantiene un mejor control del flujo de trabajo, facilita la gestión de errores y cancelaciones, y permite una ejecución más ordenada de tareas en segundo plano. Esto mejora la experiencia del usuario, reduce la carga operativa del sistema y favorece la extensibilidad del proyecto, ya que nuevos plugins pueden añadirse sin afectar el funcionamiento del núcleo ni la interfaz.

Esta organización facilita la incorporación de nuevos módulos sin modificar el núcleo, reduce la dependencia entre componentes y permite un mejor control del flujo de datos y de los resultados. La integración con servicios compartidos como DataStore, FileIOService, MeasurementService y ExportService garantiza un manejo más ordenado de la información, la persistencia y la visualización de resultados.

4.1.6. Decisiones arquitectónicas

Conforme evolucionaba el proyecto, también lo hacían las decisiones arquitectónicas necesarias para alcanzar el estado final de la implementación. Los problemas, restricciones y hallazgos identificados durante el proceso guiaron la selección y el ajuste de cada componente del diseño. A continuación se listan las decisiones arquitectónicas con mayor impacto dentro del proyecto:

Cuadro 3: ADRs resumidas: decisión, motivación y consecuencias.

ID	Decisión	Motivación	Consecuencias
ADR-01	Estilo arquitectónico de microkernel.	Permitir la incorporación de nuevas funcionalidades al sistema, incluso en tiempo de ejecución.	Sistema modular, ampliable y con bajo acoplamiento; mayor complejidad inicial.
ADR-02	Separación entre core, plugins y app.	Organizar responsabilidades y mantener claridad modular.	Facilita la comprensión, el mantenimiento y el desarrollo incremental.
ADR-03	Interfaz IPlugin.	Aplicar inversión de control para que los plugins definan únicamente su lógica.	Menor acoplamiento.
ADR-04	Carga dinámica de plugins.	Permitir la incorporación de módulos sin recompilar.	Mejor extensibilidad y colaboración entre equipos.
ADR-05	Uso de Python, PySide6 y VTK.	Utilizar librerías maduras para visualización científica y GUI.	Base sólida para análisis; requiere control cuidadoso de rendimiento y memoria.
ADR-06	Uso de DataStore.	Evitar duplicaciones y permitir la propagación reactiva de cambios.	Coherencia global y ahorro de memoria; requiere un diseño cuidadoso.
ADR-07	Kernel como orquestador central.	Ejecutar tareas pesadas sin bloquear la interfaz ni acoplar cada módulo al core.	Mayor control de concurrencia, cancelación, prioridad y trazabilidad.
ADR-08	Estructura estándar del plugin.	Unificar lógica, interfaz gráfica, metadatos y configuración.	Mejor mantenibilidad y estandarización.
ADR-09	Soporte explícito para PAC y análisis de acoplamiento.	Incorporar análisis avanzados de señales neurofisiológicas.	Mayor alcance funcional y nuevas dependencias especializadas.

4.1.7. Patrones utilizados

A continuación se presenta un resumen de los patrones arquitectónicos aplicados en el diseño de GammaLab 2.0, alineados con el estilo microkernel, el mecanismo de plugins y los servicios internos del sistema:

Cuadro 4: Patrones arquitectónicos aplicados en GammaLab 2.0.

Patrón	Problema que resuelve	Aplicación en GammaLab 2.0
Plugin	Agregar capacidades sin modificar el núcleo del sistema.	Cada plugin define un archivo <code>properties.yaml</code> y una clase que implementa <code>IPlugin</code> . El Kernel los descubre dinámicamente y los vincula en tiempo de ejecución.
IoC / Service Locator (Kernel)	Reducir el acoplamiento y centralizar los servicios compartidos.	El Kernel registra y expone servicios como <i>DataStore</i> , <i>FileIOService</i> , <i>ExportService</i> , <i>MeasurementService</i> y <i>SettingsService</i> . Los plugins acceden a estos servicios sin crear instancias propias.
Adapter (NumPy a VTK)	Integrar estructuras científicas con componentes de visualización.	El módulo <i>vtk adapters</i> convierte arrays de NumPy en <i>vtkTable</i> o estructuras compatibles para su uso en <i>vtkChartXY</i> y otros componentes VTK.
MVP / MVC en interfaz	Separar la vista de la lógica de interacción.	<i>MainWindow</i> gestiona la navegación y composición; cada plugin proporciona su interfaz mediante <code>get_widget()</code> , mientras que la lógica reside en <code>process()</code> , <code>start()</code> y <code>stop()</code> .
Caching (datos y widgets)	Reducir la latencia y la recomputación innecesaria.	Se almacenan en caché datos, resultados intermedios y widgets de plugins previamente inicializados.
Template Method (ciclo de vida)	Definir puntos de extensión uniformes.	<code>IPlugin</code> establece métodos clave (<code>initialize</code> , <code>start</code> , <code>process</code> , <code>stop</code> y <code>get_widget</code>) que todo plugin debe implementar.
Task orchestration / Executor pattern	Gestionar cálculos pesados sin bloquear la interfaz.	El Kernel centraliza la ejecución de tareas, su cola, cancelación, seguimiento del progreso y entrega de resultados.

5. Vistas Arquitectónicas (4+1)

5.1. Objetivo

Concretar **escenarios** que ejerciten decisiones y atributos de calidad de Gamma Lab 2.0. A diferencia de Gamma Lab 1.0 (sistema ya construido), Gamma Lab 2.0 documenta un sistema **parcialmente existente**: cada vista distingue explícitamente qué componentes **ya existen** en el código actual y cuáles son **nuevos**, por construir a partir de los 23 casos de uso (CU-001 a CU-023) especificados en el documento ISTAR. Cada escenario se especifica con el formato: (*Estímulo*, *Fuente*, *Entorno*, *Artefacto*, *Respuesta*, *Medida de respuesta*) y se vincula con los CU y requisitos (R-XX) que lo originan.

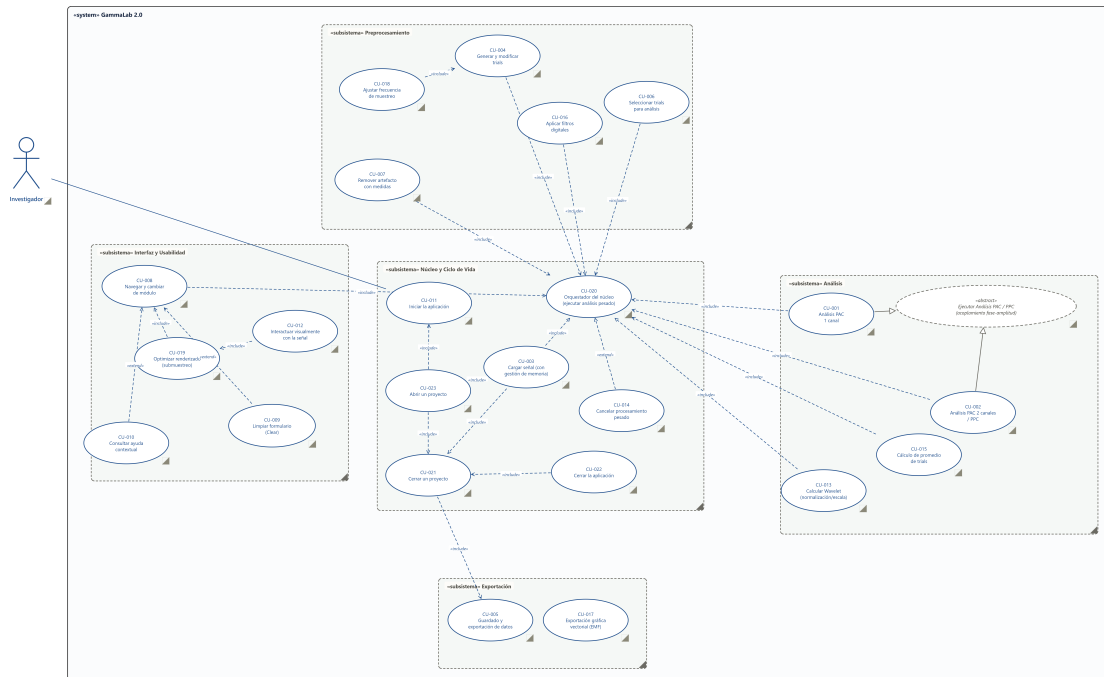


Figura 2: Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones *include/extend* se tomaron literalmente del campo “Extensiones” de cada ficha; ninguna es inferida por similitud temática.

5.2. Vista de Escenarios

La Figura 2 revela que el subsistema **Núcleo y Ciclo de Vida** — en particular CU-020, el orquestador de tareas — es el verdadero centro de gravedad arquitectónico: diez de los otros veintidós casos de uso lo incluyen explícitamente. Esa centralidad se concreta a continuación en tres escenarios representativos.

Cuadro 5: UC-01 — Calcular acoplamiento fase-amplitud (PAC) de un canal

Estímulo	El investigador configura banda de fase, rango de amplitud y trial único/promedio, y pulsa “Calcular PAC” (CU-001).
Fuente	Usuario final (investigador).
Entorno	Existe un TrialDataset activo en DataStore (CU-004/CU-006 ya ejecutados); plugin PACSingleChannelPlugin activo, categoría Analysis (nuevo).
Artefacto	PACSingleChannelPlugin (nuevo), TaskOrchestrator (nuevo), DataStore (ya existe), adaptadores VTK (ya existen).
Respuesta	El plugin delega el cómputo de la transformada wavelet al orquestador del núcleo en vez de instanciar hilos propios; al finalizar, publica el mapa fase-frecuencia, el histograma de fase y el vector de acoplamiento medio, y el resultado se renderiza en el módulo solicitante.
Medida	Indicador de progreso visible si el cómputo supera 1 s (R04); tiempos de respuesta a eventos de entrada $\leq 0,3$ s durante todo el cómputo (R55); publicación del resultado mediante escritura sincronizada, con un único propietario de escritura (R54).

Cuadro 6: UC-02 — Cancelar un cómputo pesado en curso

Estímulo	El investigador pulsa “Cancelar” mientras un análisis PAC o Wavelet está en curso (CU-014).
Fuente	Usuario final.
Entorno	Una tarea está en ejecución dentro del TaskOrchestrator (nuevo) ; hoy Kernel.execute() existe en el código pero está comentado — es un intento de despacho abandonado, no una base reutilizable.
Artefacto	TaskOrchestrator (nuevo) , hilo trabajador, IPlugin.stop() (ya existe como contrato, sin una implementación de cancelación real detrás).
Respuesta	El orquestador propaga la señal de terminación a la tarea en ejecución, garantiza el cierre y liberación del hilo trabajador aunque el cómputo siga en curso, descarta el resultado parcial sin escribirlo en el estado compartido y rehabilita la interfaz.
Medida	Liberación del hilo garantizada incluso si el cómputo no termina por sí solo (R46); si el servicio de tareas no logra terminar el proceso, fuerza el cierre del plugin y libera los recursos (excepción 2a, R70/R74).

Cuadro 7: UC-03 — Guardar y reabrir un proyecto (.glab)

Estímulo	El investigador solicita cerrar el proyecto activo y elige “Guardar” (CU-021), o abre posteriormente un archivo .glab (CU-023).
Fuente	Usuario final.
Entorno	Señal activa con trials, análisis y parámetros asociados en DataStore . Hoy existen cero referencias a “.glab” en todo el repositorio: la funcionalidad es enteramente nueva.
Artefacto	ProjectService (nuevo) , DataStore (ya existe), FileIOService (ya existe, reutilizado para recargar la señal referenciada).
Respuesta	Al guardar, ProjectService persiste señal, trials, análisis y parámetros en un archivo .glab y libera la memoria de la señal activa. Al abrir, lee el .glab , verifica que el archivo de señal referenciado siga disponible, despacha su recarga al orquestador fuera del hilo de interfaz y restaura trials y análisis mediante escritura sincronizada.
Medida	Liberación de memoria en menos de 1 s (R52); carga hasta el primer render en ≤ 2 s (R47); si el archivo de señal referenciado ya no existe, se informa la causa sin cerrar el proyecto activo (excepción, R02).

Conclusión de la vista

Los tres escenarios anclan los atributos de calidad de Gamma Lab 2.0 en situaciones verificables: rendimiento y capacidad de respuesta durante el cómputo (UC-01), robustez ante cancelación (UC-02) y persistencia confiable del estado de trabajo (UC-03). Los tres dependen de piezas que **no existen aún** en el código (**TaskOrchestrator**, **ProjectService**), lo que los convierte en criterios de aceptación directos para la implementación futura, no solo en documentación de un sistema ya construido.

5.3. Vista Lógica

5.3.1. Propósito

La vista lógica describe la descomposición funcional de Gamma Lab 2.0 en clases, interfaces y abstracciones, junto con las responsabilidades de cada elemento y las relaciones estructurales que los vinculan. Esta vista describe los elementos que lo componen, la responsabilidad que asume cada uno de ellos y la forma en que colaboran para satisfacer los requerimientos funcionales.

5.3.2. Notación

Dado que Gamma Lab 2.0 evoluciona a partir de un sistema existente, cada elemento del modelo se clasifica según su estado de implementación. Los elementos representados en color verde se encontraban implementados en el código fuente de gammaLab 1.0, mientras que los elementos representados en color rojo están planificados para gammaLab 2.0 y deben construirse para satisfacer casos de uso o requerimientos pendientes. Esta distinción permite que el modelo sirva simultáneamente como descripción del sistema pre-existente y como especificación de la arquitectura objetivo, es decir gammaLab 2.0.

5.3.3. Visión general del modelo

La estructura lógica de Gamma Lab 2.0 responde al estilo arquitectónico de microkernel, en el cual un núcleo mínimo administra un conjunto de extensiones independientes que aportan la funcionalidad del sistema. Los elementos del modelo se organizan en seis agrupaciones. El núcleo administra el ciclo de vida de las extensiones y el registro de servicios. Los servicios del sistema proveen capacidades transversales de configuración, entrada y salida de archivos, exportación, persistencia de proyectos, estado compartido y ejecución de tareas. La capa de presentación agrupa los elementos de interfaz gráfica que coordinan la interacción con el usuario. El contrato de extensión define la interfaz que todo módulo funcional debe cumplir. Las extensiones implementan las operaciones y los análisis sobre las señales. Finalmente, el modelo de datos representa las señales, sus segmentaciones y los proyectos persistidos.

La Figura 3 presenta el diagrama de clases del sistema.

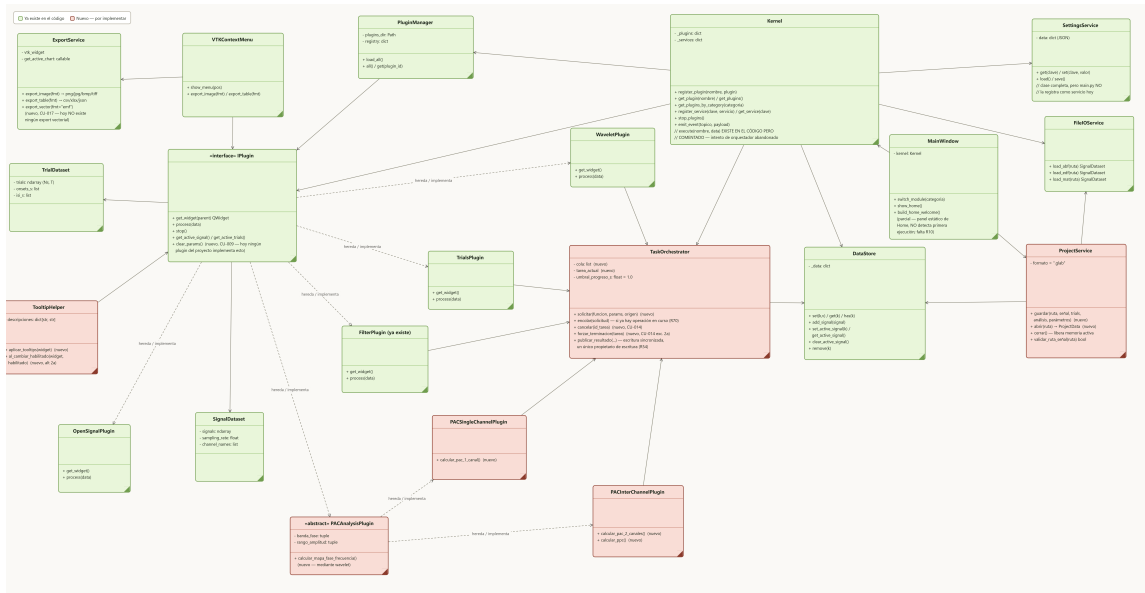


Figura 3: Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados.

5.3.4. Elementos del núcleo

Kernel El Kernel constituye el punto central de coordinación del sistema y se encuentra implementado. Mantiene dos registros internos: el registro de extensiones, indexadas por nombre, y el registro de servicios, indexados por clave. Sus operaciones permiten registrar una extensión (`register_plugin`), consultarla de forma individual o colectiva (`get_plugin` y `get_plugins`) y obtener las extensiones pertenecientes a una categoría funcional (`get_plugins_by_category`), lo cual permite a la interfaz organizar los módulos por categorías. Asimismo, ofrece operaciones para registrar y localizar servicios por clave (`register_service` y `get_service`), para detener de forma ordenada todas las extensiones al finalizar la aplicación (`stop_plugins`) y para publicar eventos dirigidos a los componentes suscritos (`emit_event`). El Kernel se asocia con el PluginManager, el SettingsService, el FileIOService, el DataStore y el TaskOrchestrator, y administra las extensiones exclusivamente a través de la interfaz IPlugin.

PluginManager El PluginManager se encuentra implementado y es responsable de descubrir, cargar e instanciar las extensiones disponibles en el directorio de extensiones del sistema. Conserva la ubicación de dicho directorio y un registro de las extensiones descubiertas. Sus operaciones permiten cargar la totalidad de las extensiones (`load_all`) y consultarlas de forma colectiva o por identificador (`all` y `get`). El PluginManager depende de la interfaz IPlugin, de modo que únicamente admite elementos que cumplan dicho contrato.

5.3.5. Servicios del sistema

SettingsService El SettingsService se encuentra implementado y gestiona la configuración persistente de la aplicación. La configuración se mantiene en memoria como un diccionario y se almacena en disco en formato JSON. El servicio ofrece operaciones para consultar y modificar valores de configuración por clave (`get` y `set`) y para cargar y almacenar la configuración (`load` y `save`).

FileIOService El FileIOService se encuentra implementado y adapta los formatos de archivo soportados al modelo de datos interno del sistema. Sus operaciones `load_abf`, `load_edf` y `load_mat` leen archivos en los formatos ABF, EDF y MAT, respectivamente, y retornan en todos los casos un objeto `SignalDataset`. De esta manera, el resto del sistema permanece independiente de la representación de las señales en disco.

ExportService El ExportService se encuentra implementado y es responsable de exportar visualizaciones y resultados tabulares a formatos externos. Mantiene una referencia al componente de visualización y una función que provee la gráfica activa en cada momento. La operación `export_image` exporta la visualización a formatos de imagen ráster (PNG, JPG, BMP y TIFF) y la operación `export_table` exporta los resultados a formatos tabulares (CSV, XLSX y JSON). Se encuentra planificada la operación `export_vector`, que permitirá la exportación a formato vectorial EMF en respuesta al caso de uso CU-017.

DataStore El DataStore se encuentra implementado y mantiene en memoria el estado compartido de la sesión de trabajo, el cual comprende las señales cargadas, la señal activa, las mediciones y los resultados de los análisis. Constituye la fuente única de verdad del estado de la aplicación. Ofrece operaciones genéricas de almacenamiento, consulta y verificación por clave (`set`, `get` y `has`), operaciones para incorporar señales (`add_signal`), para establecer, consultar y liberar la señal activa (`set_active_signal`, `get_active_signal` y `clear_active_signal`) y para eliminar entradas del estado (`remove`).

TaskOrchestrator El TaskOrchestrator tiene la responsabilidad de coordinar la ejecución de las operaciones de cómputo solicitadas por las extensiones. Su propósito es garantizar que dichas operaciones se ejecuten de forma ordenada, que puedan ser canceladas por el usuario, que se presente retroalimentación de progreso y que sus resultados se publiquen de manera consistente. Para ello mantendrá una cola de solicitudes pendientes, una referencia a la operación en ejecución y un umbral de tiempo, fijado en un segundo, a partir del cual se presentará el indicador de progreso.

La operación solicitar constituirá el punto de entrada único mediante el cual una extensión pide la ejecución de un cómputo. La operación encolar diferirá una solicitud cuando exista otra operación en curso, en cumplimiento del requerimiento R70. Las operaciones cancelar y forzar terminación permitirán, respectivamente, la cancelación de una tarea a petición del usuario y la terminación forzada de una tarea que no responda a la cancelación, conforme al caso de uso CU-014 y su excepción 2a. Por último, la operación publicar resultado escribirá los resultados en el estado compartido de forma sincronizada. El TaskOrchestrator será el único componente autorizado para escribir en el DataStore, en cumplimiento del requerimiento R54.

ProjectService El ProjectService se encuentra implementado y gestiona la persistencia del espacio de trabajo en archivos de proyecto con extensión `glab`. Cada archivo de proyecto es un manifiesto en formato JSON que registra la señal de origen, las mediciones realizadas, la configuración de la segmentación en ensayos y el estado de los análisis. Durante el guardado, el archivo de señal se copia a la carpeta del proyecto, de modo que el proyecto resulta autocontenido y puede reabrirse en otro equipo. Durante la apertura, el servicio verifica la disponibilidad del archivo de señal referenciado, lo recarga mediante el FileIOService y restaura el estado de la sesión en el DataStore.

El servicio mantiene la carpeta del proyecto, la ruta del archivo de proyecto, el nombre del proyecto, un indicador de cambios pendientes de guardar y una función de notificación que informa

a la interfaz sobre la apertura, el guardado, el cambio de nombre o el cierre del proyecto. Sus operaciones principales permiten guardar el proyecto en su ubicación actual (save), guardarlo en una nueva ubicación (save_as), abrir y restaurar un proyecto existente (open), y cerrar el proyecto activo liberando su estado (close). Adicionalmente, ofrece operaciones para asignar el nombre del proyecto (set_project_name), registrar la existencia de cambios pendientes (mark_dirty), verificar si existe un proyecto abierto (is_open) y generar un nombre predeterminado a partir del nombre de la señal (default_project_name). El ProjectService depende del FileIOService, del DataStore y del Kernel, y produce y consume objetos de tipo ProjectData.

5.3.6. Elementos de presentación

MainWindow La MainWindow se encuentra parcialmente implementada y constituye el contenedor principal de la interfaz gráfica. Es responsable de la navegación entre los módulos funcionales, de la presentación de la pantalla de inicio y de las operaciones de proyecto disponibles en el menú Archivo. Mantiene referencias al Kernel y al ProjectService. La operación switch_module presenta las extensiones correspondientes a la categoría seleccionada por el usuario, la operación (show_home) presenta la pantalla de inicio y la operación (build_home_welcome) construye el panel de bienvenida. Esta última se encuentra pendiente de incorporar la detección de la primera ejecución de la aplicación, requerida por el requerimiento R10.

VTKContextMenu El VTKContextMenu se encuentra implementado y provee el menú contextual asociado a las visualizaciones. Sus operaciones permiten presentar el menú en la posición indicada por el usuario (show_menu) y solicitar la exportación de la visualización como imagen o como tabla (export_image y export_table). Este elemento delega la exportación en el ExportService y opera sobre los componentes visuales provistos por las extensiones.

TooltipHelper El TooltipHelper se encuentra planificado y tendrá la responsabilidad de proveer ayuda contextual sobre los controles de la interfaz. Mantendrá un diccionario de descripciones asociadas a cada control. La operación (aplicar_tooltips) asignará dichas descripciones a los controles de un componente visual y la operación (al_cambiar_habilitado) actualizará la descripción de un control cuando cambie su disponibilidad, de modo que el usuario conozca el motivo por el cual un control se encuentra deshabilitado.

5.3.7. Contrato de extensión

IPlugin IPlugin es la interfaz que define el contrato que toda extensión debe cumplir para integrarse al sistema, y constituye el mecanismo principal de extensibilidad de la arquitectura. Se encuentra implementada y establece las siguientes operaciones: get_widget, que provee la interfaz gráfica de la extensión; process, que ejecuta el procesamiento propio de la extensión; stop, que detiene la extensión y libera sus recursos; y get_active_signal y get_active_trials, que proporcionan acceso a los datos de trabajo. IPlugin depende de las clases SignalDataset y TrialDataset.

Se encuentra planificada la incorporación al contrato de la operación clear_params, cuya finalidad es restablecer los parámetros de una extensión a sus valores predeterminados, conforme al caso de uso CU-009. Actualmente esta operación se encuentra implementada de forma individual en las extensiones FilterPlugin, TrialsPlugin y ArtifactRemovePlugin, pero aún no forma parte de la interfaz común.

5.3.8. Extensiones

Todas las extensiones del sistema realizan la interfaz IPlugin. La Tabla 8 resume las extensiones representadas en el modelo, su estado de implementación y su responsabilidad.

Cuadro 8: Extensiones representadas en la vista lógica.

Elemento	Responsabilidad
OpenSignalPlugin	Selección y carga de señales desde archivo.
FilterPlugin	Filtrado en frecuencia de la señal.
WaveletPlugin	Análisis en tiempo y frecuencia mediante la transformada wavelet.
TrialsPlugin	Segmentación de la señal en ensayos.
PACAnalysisPlugin	Comportamiento común del análisis de acoplamiento entre fase y amplitud (PAC).
PACSingleChannelPlugin	Cálculo del acoplamiento entre fase y amplitud dentro de un mismo canal.
PACInterChannelPlugin	Cálculo del acoplamiento entre fase y amplitud y de la consistencia de fase por pares (PPC) entre dos canales.

PACAnalysisPlugin es una clase abstracta que concentra los parámetros y el procedimiento comunes a todo análisis de acoplamiento entre fase y amplitud. Sus atributos definen la banda de frecuencia de la fase y el rango de frecuencia de la amplitud, y su operación `calcular_mapa_fase_frecuencia` obtiene el mapa de fase y frecuencia mediante la transformada wavelet. Las clases PACSingleChannelPlugin y PACInterChannelPlugin especializan este comportamiento. La primera incorpora la operación `calcular_pac_1_canal`, que calcula el acoplamiento dentro de un mismo canal, y la segunda incorpora las operaciones `calcular_pac_2_canales` y `calcular_ppc`, que calculan el acoplamiento y la consistencia de fase entre dos canales.

5.3.9. Modelo de datos

SignalDataset SignalDataset se encuentra implementada y representa una señal electrofisiológica multicanal. Sus atributos contienen la matriz de muestras de la señal, la frecuencia de muestreo expresada en hercios y los identificadores de los canales.

TrialDataset TrialDataset se encuentra implementada y representa una señal segmentada en ensayos. Sus atributos contienen la matriz de ensayos, cuyas dimensiones corresponden al número de muestras por ensayo y al número de ensayos, los instantes de inicio de cada ensayo expresados en segundos y los intervalos entre estímulos expresados en segundos.

ProjectData ProjectData es una estructura de datos implementada que representa en memoria el contenido de un archivo de proyecto. En la implementación actual se materializa como un diccionario serializado en formato JSON, producido por la operación `save` y retornado por la operación `open` del ProjectService. Sus atributos registran la versión del esquema del formato de proyecto, el nombre del proyecto, la fecha y hora del último guardado, la información de la señal de origen (nombre de archivo, formato, nombre y frecuencia de muestreo), las mediciones

realizadas, la configuración de la segmentación en ensayos y los parámetros y el estado de las extensiones de análisis. La versión del esquema permite detectar incompatibilidades entre versiones del formato al momento de abrir un proyecto.

5.3.10. Relaciones estructurales

La Tabla 9 resume las relaciones entre los elementos del modelo lógico.

Cuadro 9: Relaciones estructurales de la vista lógica.

Origen	Tipo	Destino	Descripción
Kernel	Asociación	PluginManager	Delega la carga de extensiones.
Kernel	Asociación	SettingsService, FileIOService, DataStore, TaskOrchestrator	Registra y expone los servicios del sistema.
Kernel, PluginManager	Dependencia	IPlugin	Administran las extensiones a través de su contrato.
MainWindow	Asociación	Kernel, ProjectService	Accede a las extensiones, a los servicios y a las operaciones de proyecto.
ProjectService	Dependencia	FileIOService, DataStore	Recarga la señal y restaura el estado de la sesión.
ProjectService	Dependencia	ProjectData	Produce y consume el contenido del proyecto.
VTKContextMenu	Dependencia	ExportService	Delega la exportación.
VTKContextMenu, TooltipHelper	Dependencia	IPlugin	Operan sobre los componentes visuales de las extensiones.
IPlugin	Dependencia	SignalDataset, TrialDataset	Manipula el modelo de datos.
Extensiones concretas y PACAnalysisPlugin	Realización	IPlugin	Implementan el contrato de extensión.
PACSingleChannelPlugin, PACInterChannelPlugin	Generalización	PACAnalysisPlugin	Especializan el análisis de acoplamiento.
Extensiones de análisis	Dependencia	TaskOrchestrator	Solicitan la ejecución de operaciones de cómputo.
TaskOrchestrator	Asociación	DataStore	Publica los resultados en el estado compartido.

5.3.11. Justificación de las decisiones

La separación entre el núcleo y las extensiones mediante la interfaz IPlugin favorece la extensibilidad del sistema, ya que permite incorporar nuevos análisis, como los de acoplamiento entre fase y amplitud, sin modificar el código existente, en conformidad con el principio de abierto y cerrado.

La asignación exclusiva de la escritura sobre el estado compartido al TaskOrchestrator garantiza la consistencia del estado de la aplicación, al eliminar las condiciones de carrera entre operaciones concurrentes, conforme al requerimiento R54.

La centralización de la ejecución de los cálculos en el TaskOrchestrator mejora la capacidad de respuesta del sistema y el control que ejerce el usuario sobre él, puesto que habilita la ejecución ordenada de las operaciones (R70), su cancelación (CU-014) y la presentación de progreso, sin que cada extensión deba gestionar sus propios hilos de ejecución.

La concentración de la lectura de archivos en el FileIOService asegura la independencia del sistema respecto de los formatos de señal, y confina a un único elemento el impacto de incorporar nuevos formatos.

La persistencia de proyectos mediante el ProjectService y la estructura ProjectData produce proyectos autocontenidos y versionados. La inclusión de una copia de la señal de origen permite reabrir el trabajo en otro equipo, y la versión del esquema permite detectar incompatibilidades entre versiones del formato.

Por último, la jerarquía encabezada por PACAnalysisPlugin promueve la reutilización, al evitar la duplicación del cálculo del mapa de fase y frecuencia entre las variantes de análisis de uno y de dos canales.

5.3.12. Trazabilidad con los requerimientos

La Tabla 10 relaciona los elementos y operaciones del modelo lógico con los casos de uso y requerimientos que los originan.

Cuadro 10: Trazabilidad entre elementos de la vista lógica y requerimientos.

Elemento u operación	Caso de uso o requerimiento
IPlugin, operación clear_params	CU-009
TaskOrchestrator, operación cancelar	CU-014
TaskOrchestrator, operación forzar_terminacion	CU-014, excepción 2a
ExportService, operación export_vector	CU-017
ProjectService, operaciones save, save_as y close	CU-021, R07
ProjectService, operación open, y ProjectData	CU-023
TaskOrchestrator, operación publicar_resultado	R54
TaskOrchestrator, operación encolar	R70
MainWindow, operación build_home_welcome	R10
PACSingleChannelPlugin y PACInterChannelPlugin	CU-001 y CU-002

5.4. Vista de Procesos

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondition declaradas por las propias 23 fichas (p. ej. CU-006 exige trials generados por CU-004, que a su vez exige una señal cargada por CU-003) — no es una fusión literal de los 23 flujos detallados, que sería ilegible.

El flujo de la Figura 4 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo y descubre los plugins vía YAML (CU-011); el registro del servicio de tareas en este paso es nuevo, el resto ya existe en `main.py`.
2. **Apertura o carga.** Según si el investigador abre un `.glab` (CU-023, nuevo) o carga una señal desde Home vacío (CU-003, ya existe vía `FileIOService`), la señal activa termina publicándose en `DataStore`.

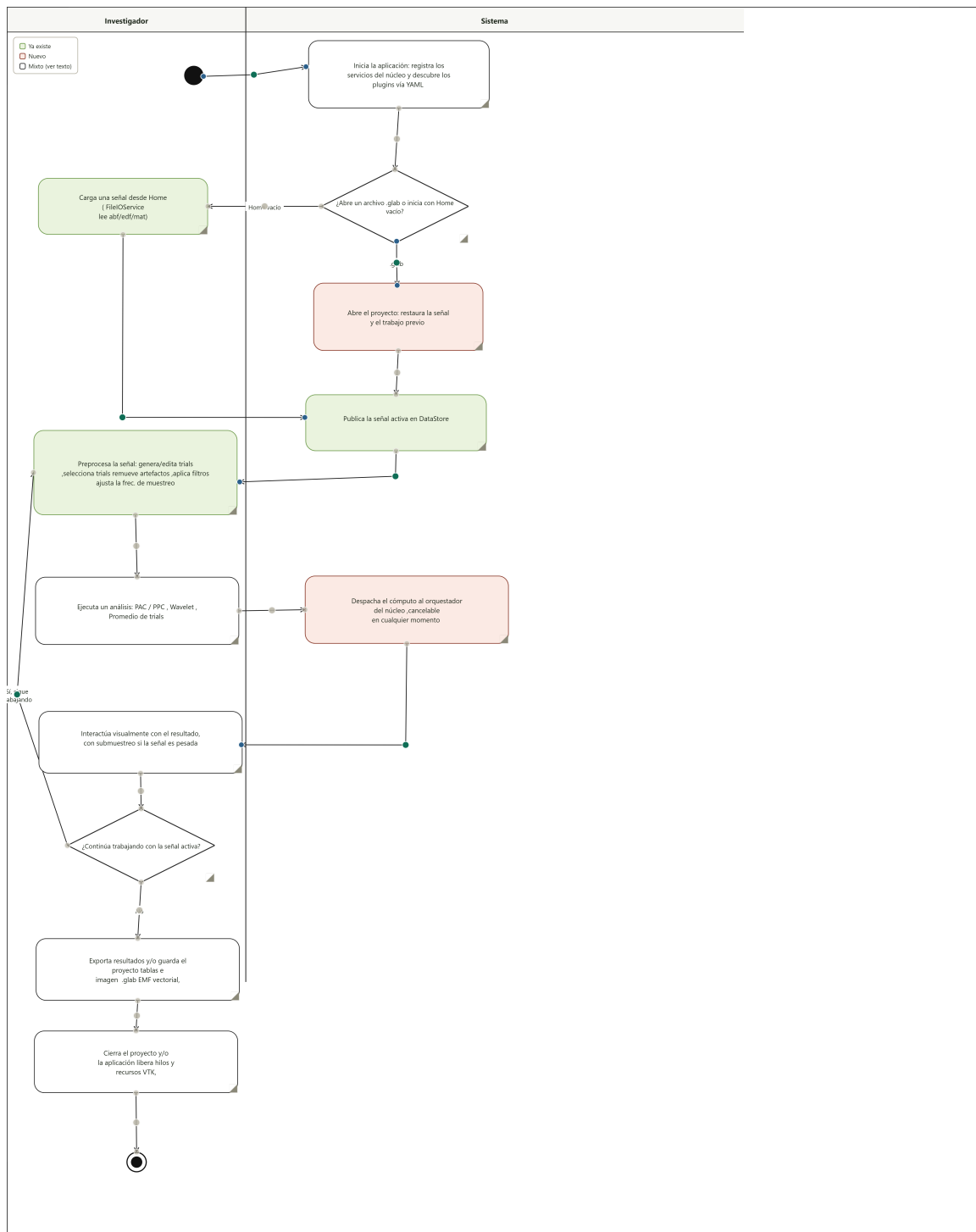


Figura 4: Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.

3. **Preprocesamiento.** Generación/edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de frecuencia de muestreo (CU-004, CU-006, CU-007, CU-016, CU-018) — los cuatro plugins ya existen en el código.
4. **Análisis.** PAC/PPC (CU-001, CU-002, nuevos), Wavelet (CU-013), promedio de trials (CU-015).
5. **Orquestación.** Todo cómputo pesado se despacha al orquestador del núcleo (CU-020, nuevo), cancelable en cualquier momento (CU-014, nuevo).
6. **Visualización.** Interacción visual con el resultado, con submuestreo si la señal es pesada (CU-012 parcial + CU-019, nuevo).
7. **Iteración o cierre.** Si el investigador continúa trabajando, el flujo retorna al preprocesamiento; de lo contrario, exporta y/o guarda el proyecto (CU-005 parcial, CU-017 nuevo) y cierra el proyecto y/o la aplicación (CU-021 nuevo, CU-022 parcial).

A diferencia de Gamma Lab 1.0, los casos de uso de navegación (CU-008), limpieza de formulario (CU-009, 0 % implementado) y ayuda contextual (CU-010, cobertura parcial en solo 2 de más de 15 plugins) son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en lugar de forzarlas dentro del flujo principal.

5.5. Vista Física

La vista física describe el entorno de despliegue local de Gamma Lab 2.0. A diferencia de Gamma Lab 1.0, aquí se documenta explícitamente la superficie completa de entrada/salida del sistema, incluyendo los artefactos que aún no existen.

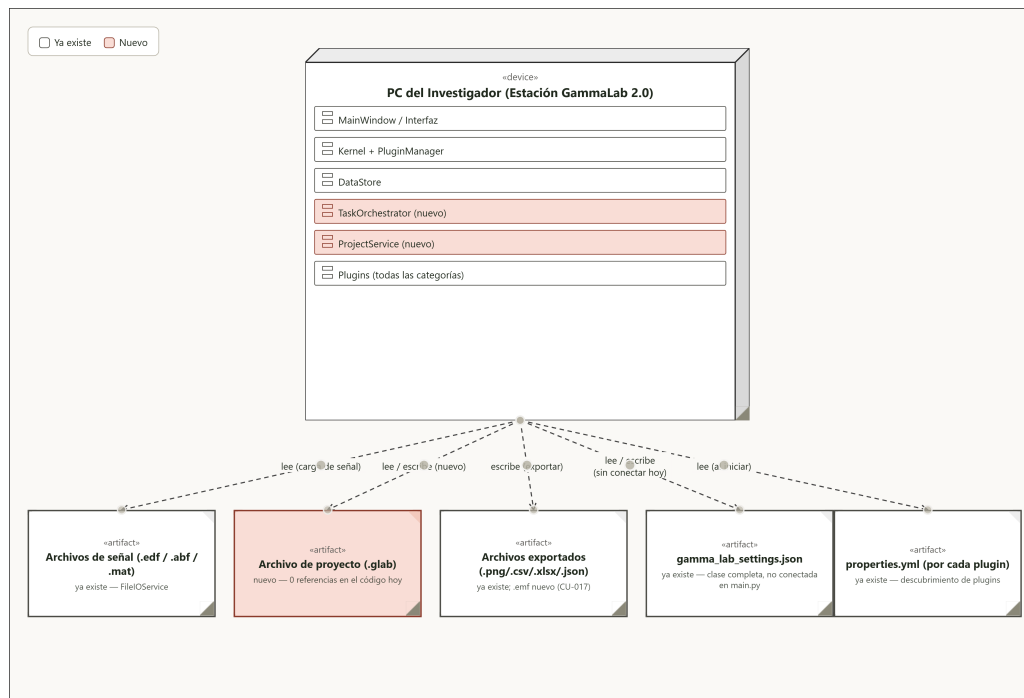


Figura 5: Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.

La Figura 5 muestra el despliegue típico:

- Un único nodo *PC del Investigador*: no existe arquitectura cliente-servidor en ninguna de las 23 fichas.
- **Archivos de señal (.edf/.abf/.mat)**: ya existe, leídos por FileIOService.
- **Archivo de proyecto (.glab)**: nuevo — cero referencias en el código actual.

- **Archivos exportados** (.png/.csv/.xlsx/.json): ya existe vía `ExportService`; el formato vectorial .emf (CU-017) es nuevo.
- **gamma_lab_settings.json**: la clase `SettingsService` ya existe, pero **no** está conectada en el arranque (`main.py`) hoy.
- **properties.yml** (por cada plugin): ya existe, sostiene el descubrimiento automático de plugins.

6. Atributos de calidad

6.1. Escenarios de calidad

6.1.1. Escenario de Calidad 1: Integración de un Plugin Externo

Cuadro 11: Escenario de calidad 1 — Integración de un plugin externo

Aspecto	Descripción
Atributo de calidad	Soporte
Requisito	El sistema debe permitir integrar plugins externos sin recompilar el núcleo y sin afectar módulos existentes.
Fuente del estímulo	Desarrollador externo
Estímulo	Se agrega un nuevo plugin a la carpeta de Plugins siguiendo el formato definido por el Kernel.
Artefacto	Módulo core del sistema (Kernel) y módulo de plugins.
Ambiente	Sistema en operación normal con múltiples plugins ya registrados.
Respuesta	El sistema detecta, valida y registra el plugin, exponiéndolo en la interfaz de usuario.
Medida de Respuesta	Plugin visible en la UI en menos de 1 minuto. Ningún plugin existente debe fallar tras la carga.
Trazabilidad	NFR-S-05 (R77–R78)

6.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño

Cuadro 12: Escenario de calidad 2 — Carga de señal de gran tamaño

Aspecto	Descripción
Atributo de calidad	Rendimiento
Requisito	El sistema debe gestionar señales de gran tamaño sin bloqueos ni degradación significativa del rendimiento.
Fuente del estímulo	Usuario final
Estímulo	Se carga un archivo ABF/EDF de gran tamaño (ej. 1–2 GB, múltiples canales).
Artefacto	FileIOService, DatasetService (DataStore), servicio de tareas del núcleo, interfaz de usuario.
Ambiente	Computador con carga moderada y otros procesos del sistema activos.
Respuesta	La señal se carga correctamente, sin duplicación innecesaria de datos y sin bloqueos de la interfaz; la lectura corre en el servicio de tareas, fuera del hilo de la UI.
Medida de Respuesta	Visualización inicial en menos de 8 segundos para este caso de estrés (el umbral base de carga es ≤ 2 s para el archivo típico de referencia). Uso de CPU menor a 80 %. Sin cierres inesperados ni latencias mayores a 5 segundos.
Trazabilidad	NFR-P-03 (R48), NFR-P-14 (R72), NFR-P-10 (R56)

6.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución

Cuadro 13: Escenario de calidad 3 — Manejo de errores en tiempo de ejecución

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe manejar excepciones en los plugins sin comprometer la estabilidad del sistema.
Fuente del estímulo	Sistema o entorno
Estímulo	Un plugin genera una excepción interna durante un análisis (FFT, PSD, Wavelet, ERP, PAC, etc.).
Artefacto	Módulo de plugins, Kernel, servicios asociados a cada plugin.
Ambiente	Análisis en curso con señales cargadas y otros plugins activos.
Respuesta	El sistema captura la excepción, informa al usuario y evita un cierre inesperado de la aplicación, manteniendo el Kernel operando de manera adecuada.
Medida de Respuesta	0 % de pérdida de datos. Continuidad del sistema garantizada. Mensaje de error mostrado en menos de 2 segundos.
Trazabilidad	R47

6.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas

Cuadro 14: Escenario de calidad 4 — Exportación de datos o gráficas

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe exportar gráficos y datos sin errores, incluso si se presentan fallas externas por disco, permisos, memoria u otros motivos.
Fuente del estímulo	Usuario final
Estímulo	El usuario exporta una tabla, medición o gráfico en PNG/CSV/JSON.
Artefacto	ExportService (registrado en el núcleo por inversión de control), interfaz de usuario, sistema de archivos.
Ambiente	Sistema operando con múltiples análisis realizados previamente sin guardar.
Respuesta	El archivo se genera correctamente o, en caso de error, se notifica al usuario sin afectar el estado del sistema.
Medida de Respuesta	Tiempo de exportación menor a 3 segundos. 0 % de pérdida de datos. Notificación clara ante error.
Trazabilidad	NFR-P-07 (R51), NFR-P-12 (R57), NFR-S-03 (R70)

6.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica

Cuadro 15: Escenario de calidad 5 — Operación por un usuario sin experiencia técnica

Aspecto	Descripción
Atributo de calidad	Usabilidad
Requisito	El sistema debe ser utilizable por un usuario con experiencia en neurociencias o análisis de señales, pero sin conocimientos profundos del software.
Fuente del estímulo	Usuario final
Estímulo	El usuario utiliza un plugin complejo como Wavelet, PSD o PAC por primera vez.
Artefacto	Interfaz de usuario, ayudas contextuales, parámetros preconfigurados.
Ambiente	Uso sin acompañamiento técnico.
Respuesta	El usuario puede completar el análisis apoyándose en la interfaz, con guía clara y sin errores operativos.
Medida de Respuesta	Análisis realizado en menos de 5 minutos. Bajo número de consultas al manual de usuario.
Trazabilidad	NFR-U-07 (R42), NFR-U-08 (R43), NFR-U-05 (R10), NFR-U-09 (R44)

6.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas

Cuadro 16: Escenario de calidad 6 — Rendimiento con múltiples señales cargadas

Aspecto	Descripción
Atributo de calidad	Rendimiento
Requisito	El sistema debe mantener tiempos de respuesta estables aun cuando varias señales se encuentren cargadas simultáneamente dentro de la misma ProjectSession.
Fuente del estímulo	Usuario final
Estímulo	El usuario carga entre 2 y 4 señales de gran tamaño y ejecuta diferentes análisis en cada una.
Artefacto	DataStore, gestor de vistas, motor de análisis.
Ambiente	Sistema operando con alto volumen de datos en memoria.
Respuesta	El sistema reutiliza datos sin duplicarlos, mantiene tiempos de acceso aceptables y evita bloqueos en la interfaz.
Medida de Respuesta	Cambio entre señales cargadas en menos de 1 segundo. Sin bloqueos ni cierres inesperados. Uso de memoria dentro del 70–75 %.
Trazabilidad	NFR-P-06 (R51), NFR-P-01/P-08 (R11, R53)

6.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles

Cuadro 17: Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe detectar archivos corruptos, incompletos o incompatibles sin comprometer la estabilidad del programa.
Fuente del estímulo	Usuario final o sistema externo
Estímulo	El usuario intenta cargar un archivo ABF/EDF/MAT con estructura inválida o con metadatos corruptos.
Artefacto	FileIOService, Kernel, sistema de alertas.
Ambiente	Sistema en operación normal con señales previamente cargadas.
Respuesta	El sistema captura el error, notifica al usuario, cancela la operación de carga y mantiene intacta la sesión actual.
Medida de Respuesta	Mensaje de error mostrado en menos de 2 segundos. 0 % de pérdida de datos o señales cargadas previamente. El sistema continúa operativo sin fallos.
Trazabilidad	FR-IO-04

6.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas

Cuadro 18: Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas

Aspecto	Descripción
Atributo de calidad	Confiabilidad y Usabilidad
Requisito	El sistema debe garantizar que las gráficas generadas correspondan fielmente a los datos y parámetros seleccionados por el usuario.
Fuente del estímulo	Usuario final
Estímulo	El usuario modifica parámetros como canal, ventana temporal, trials seleccionados o rango de frecuencias.
Artefacto	Interfaz de usuario, motor de visualización, servicios de análisis.
Ambiente	Sistema ejecutando análisis consecutivos sobre diferentes configuraciones.
Respuesta	La gráfica generada refleja exactamente los datos filtrados o seleccionados, sin desajustes ni inconsistencias con los parámetros ingresados.
Medida de Respuesta	Actualización gráfica en menos de 3 segundos. 0 diferencias entre los datos mostrados y los procesados. Consistencia confirmada entre ejecuciones sucesivas.
Trazabilidad	NFR-R-04 (R73), NFR-U-02 (R03)

6.2. Reglas de compatibilidad y versionado

- **Versionado de la aplicación.** GammaLab 2.0 adopta versionado semántico MAJOR. MINOR. PATCH: MAJOR rompe `.glab` o el contrato `IPlugin`; MINOR agrega capacidades retrocompatibles; PATCH corrige sin alterar contratos públicos. `main.py` expone esta versión en una constante única, visible en un diálogo “Acerca de” y embebida como metadato en cada archivo `.glab` generado.
- **Contrato de plugins (IPlugin).** Cambios aditivos (nuevos métodos opcionales, nuevos campos con valor por defecto) no rompen compatibilidad. Cambios que alteren la firma de `initialize`, `start`, `stop`, `process` o `get_widget` elevan la versión MAJOR del contrato.
- **Metadatos properties.yml.** Los campos requeridos (`id`, `name`, `category`, `subcategory`, `version`, `icon`, `logic_class`, `ui_class`) deben existir siempre – exigido por `REQUIRED_KEYS` en `loader.py` – y añadir campos opcionales es compatible. El campo `version` declara el MAJOR de `IPlugin` contra el que se escribió el plugin; `discover()` rechaza el plugin si no coincide con el MAJOR del núcleo en ejecución.
- **Formato de proyecto .glab.** `ProjectService` persiste el espacio de trabajo – señal activa, trials y análisis – en archivos `.glab`. Cada `.glab` embebe un `schema_version` propio: un incremento MINOR se mantiene legible por versiones futuras de `ProjectService`, y un incremento MAJOR rechaza la apertura con un mensaje explícito al usuario en vez de migrar implícitamente.
- **Dependencias externas.** `requirements.txt` fija cada librería con versión exacta (`==`), sin rangos abiertos (Tabla 19). Actualizar VTK, SciPy, PyWavelets o NumPy exige re-ejecutar completas las suites `plugins_test` y `performance_test` antes de aceptar el cambio.
- **Formatos de señal externos (EDF/ABF/BDF/MAT).** La compatibilidad se ancla a la versión de la librería lectora (`pyabf`, `pyedflib`), no al formato en sí. Un archivo no

interpretable por esa versión se reporta como incompatible, sin remuestreo ni heurísticas de recuperación implícitas: un EDF con canales a distinta frecuencia de muestreo falla con un mensaje claro, en vez de intentar corregirse automáticamente.

- **Estabilidad de módulos internos.** Todo renombramiento o reubicación de un módulo público de `core/` actualiza en el mismo cambio a todos sus consumidores, tests incluidos, y un flujo de CI corre `pytest` en cada *push/pull request*. Motivo: un renombramiento previo de `fileio.py` a `fileio_service.py` (con `TrialDataset` movido a `core/model/`) rompió silenciosamente los 8 archivos de `plugins_test` al no actualizarse en el mismo cambio.

Cuadro 19: Versiones de dependencias externas ancladas en `requirements.txt`

Librería	Versión	Rol
<code>numpy</code>	2.3.4	Cálculo numérico vectorizado
<code>scipy</code>	1.16.2	Filtros, PSD (Welch)
<code>PyWavelets</code>	1.9.0	Transformada Wavelet Continua
<code>vtk</code>	9.5.2	Renderizado científico 2D/3D
<code>PyQt5 / PyQt5-Qt5</code>	5.15.11 / 5.15.2	Interfaz gráfica
<code>pyabf</code>	2.3.8	Lectura de archivos <code>.abf</code>
<code>pyEDFlib</code>	0.1.42	Lectura de archivos <code>.edf</code>
<code>h5py</code>	3.15.1	Soporte de formatos basados en HDF5
<code>pytest</code>	8.4.2	Framework de pruebas

6.3. Métricas y plan de verificación

La Tabla 20 formaliza la “Medida de Respuesta” de cada escenario de calidad (sección 5.1) en una métrica verificable, junto con su método de medición y el criterio de aceptación correspondiente.

Cuadro 20: Métricas, método de verificación y criterio de aceptación por atributo de calidad

Atributo	Métrica	Método	Criterio de aceptación
Extensibilidad (Esc. 1)	Tiempo de aparición de un plugin nuevo; plugins existentes que fallan	Agregar un plugin válido y correr <code>plugins_test</code>	<1 min; 0 plugins fallan (R77–R78)
Rendimiento – carga grande (Esc. 2)	Tiempo a primer render; uso de CPU	<code>performance_test</code> ; medido: 78.4 ms (1.8 M muestras)	≤8 s estrés, ≤2 s típico; CPU <80 % (R48/R72/R56)
Confiabilidad – errores en ejecución (Esc. 3)	% pérdida de datos; tiempo del mensaje	Inyección de fallos dentro de <code>process()</code>	0 % pérdida; mensaje <2 s; sistema operativo
Confiabilidad – exportación (Esc. 4)	Tiempo de exportación; error silencioso	<code>test_export.py</code>	<3 s; 0 % pérdida; notificación clara (R51/R57/R70)
Usabilidad (Esc. 5)	Tiempo de análisis sin acompañamiento	Prueba de usabilidad con usuarios reales	<5 min; bajas consultas al manual (R42–R44)

Continúa en la siguiente página

Atributo	Métrica	Método	Criterio de aceptación
Rendimiento – múltiples señales (Esc. 6)	Tiempo de cambio; % de memoria	Benchmark de DataStore	<1 s cambio; memoria 70–75 % (R11, R51, R53)
Confiabilidad – archivos corruptos (Esc. 7)	Tiempo del error; continuidad de sesión	<code>test_fileio_abf/edf.py</code>	Mensaje <2 s; 0 % pérdida previa (FR-IO-04)
Confiabilidad y Usabilidad (Esc. 8)	Tiempo de actualización; diferencias	Revisión de consistencia entrada-salida	<3 s; 0 diferencias (R73, R03)
Mantenibilidad	% de cobertura automática	<code>pytest --cov sobre core/plugins</code>	≥80 % de cobertura
Rendimiento – no regresión vs. 1.0	Factor vs. línea base	<code>compare_report.py</code>	Nada empeora; render sin vectorizar mejora ≥10×
Portabilidad	Éxito de arranque del <code>.exe</code>	Instalación en Windows limpio sin Python	Arranque exitoso, sin configuración manual
Robustez – cancelación (UC-02)	Liberación del hilo tras cancelar	Prueba forzada sobre <code>TaskOrchestrator</code>	Liberación garantizada (R46, R70, R74)

6.4. Plan de evidencia

1. **Fixture ABF real y referencias MATLAB.** `test/data/17308005.abf` más los 8 CSV de referencia en `plugins_test/` sustentan la corrección numérica exigida por Extensibilidad (Esc. 1) y por las métricas de Mantenibilidad.
2. **Suite de benchmarks versionada.** `performance_test/`, con resultados persistidos por etiqueta en `perf_results.json` (p.ej. `gammalab1_baseline_2026-08-16`), es la evidencia reproducible de Rendimiento y de la comparación de no regresión frente a 1.0.
3. **Auditorías documentales.** Los informes `reporte_tests.tex` e `informe_rendimiento.tex` (`docs/tests documents/`) registran el estado de la suite de pruebas y los hallazgos de rendimiento.
4. **Pruebas de usabilidad con usuarios reales.** `docs/pruebas-usabilidad/` sustenta el criterio de aceptación de Usabilidad (Esc. 5).
5. **Prueba de instalación limpia.** Arranque del ejecutable generado por PyInstaller en un Windows 10/11 sin Python previamente instalado, como evidencia de Portabilidad.
6. **Registro de cancelación forzada.** La prueba de cancelación forzada sobre el `TaskOrchestrator` bajo carga es la evidencia de Robustez (UC-02).

6.5. Conclusión

Los escenarios de calidad, las reglas de compatibilidad y la tabla de métricas convierten los atributos de calidad de la sección 3.2 en criterios operativos y verificables, no en aspiraciones genéricas. El plan de evidencia de la sección 6.4 sostiene estos criterios con artefactos reproducibles: fixtures reales validados contra una referencia de MATLAB, benchmarks versionados por etiqueta, auditorías documentales, y pruebas de usabilidad y de instalación. Junto con las reglas

de versionado de la sección 6.2, esta estructura garantiza que la arquitectura de GammaLab 2.0 evalúe, documente y demuestre sus atributos de calidad de forma reproducible en cada nueva versión del sistema.

7. Anexos

7.1. Anexo A — Relación con el SRS

En particular:

-

7.2. Anexo B — Relación con el SDD

Este anexo resume dicha relación:

-

7.3. Anexo C — Trazabilidad global de artefactos

7.4. Anexo D — Diagramas y tablas de referencia

8. Referencias

Referencias

- [1] Python Software Foundation. *Python 3 — The Python Standard Library*. Disponible en: <https://docs.python.org/3/>.
- [2] Riverbank Computing. *PyQt5 Reference Guide*. Disponible en: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
- [3] The Qt Company. *Qt 5 — Style Sheets Reference*. Disponible en: <https://doc.qt.io/qt-5/stylesheet-syntax.html>.
- [4] Kitware, Inc. *VTK — Visualization Toolkit Documentation*. Disponible en: <https://vtk.org/> y <https://docs.vtk.org/>.
- [5] Kitware, Inc. *vtkChartXY Class Reference*. Disponible en: <https://docs.vtk.org/en/latest/api/python/vtkmodules.vtkChartsCore.html#vtkmodules.vtkChartsCore.vtkChartXY>.
- [6] NumPy Developers. *NumPy Documentation*. Disponible en: <https://numpy.org/doc/>.
- [7] Scott W. Harden. *pyABF — Read and Analyze ABF Files with Python*. Disponible en: <https://pyabf.readthedocs.io/>.
- [8] Holger Nahrstaedt et al. *pyEDFlib — Python library to read/write EDF+/BDF+ files*. Disponible en: <https://pyedflib.readthedocs.io/>.

- [9] Oren Ben-Kiki, Clark Evans, Ingy döt Net. *YAML Ain't Markup Language (YAML) Specification*. Disponible en: <https://yaml.org/spec/>.
- [10] P. B. Kruchten. *The 4+1 View Model of Architecture*. IEEE Software, vol. 12, no. 6, pp. 42–50, 1995. DOI: <https://doi.org/10.1109/52.469759>.
- [11] Len Bass, Paul Clements, Rick Kazman. *Software Architecture in Practice* (3rd ed.). Addison–Wesley, 2012. ISBN 978-0-321-81573-6.
- [12] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Robert Nord, Judith Stafford. *Documenting Software Architectures: Views and Beyond* (2nd ed.). Addison–Wesley, 2010. ISBN 978-0-321-55268-6.
- [13] Rick Kazman, Mark Klein, Paul Clements. *ATAM: Method for Architecture Evaluation*. CMU/SEI-2000-TR-004, Software Engineering Institute, 2000.
- [14] Bob Kemp, Alpo Värri, Agostinho Rosa, Kim D. Nielsen, John Gade. *A simple format for exchange of digitized polygraphic recordings*. Electroencephalography and Clinical Neurophysiology, 82(5), 1992, pp. 391–393. DOI: [https://doi.org/10.1016/0013-4694\(92\)90009-7](https://doi.org/10.1016/0013-4694(92)90009-7).
- [15] P. L. Nunez, R. Srinivasan. *Electric Fields of the Brain: The Neurophysics of EEG* (2nd ed.). Oxford University Press, 2006.
- [16] Equipo Gamma Lab. *Software Architecture Document (SAD), v1.0*. Este documento.

Nota: Las URLs señaladas se incluyen para consulta rápida. En publicación impresa, usar DOI/ISBN cuando aplique.